

OOT_判断题

一、判断题例题

A. 设计模式判断题

1. 策略模式适合将一组可替换的算法或规则封装起来，并让上下文对象在运行时选择其中一种策略。（ ）

答案：对

解析： Strategy 的核心就是封装可替换算法。

2. 策略模式的主要目的是保证一个类在系统中只有一个实例。（ ）

答案：错

解析： 这是 Singleton 的目的，不是 Strategy。

3. 观察者模式适合一个对象状态变化后，需要通知多个依赖对象的场景。（ ）

答案：对

解析： Observer 适合一对多通知。

4. 观察者模式要求 Subject 必须知道每一个 ConcreteObserver 的具体类型。（ ）

答案：错

解析： Subject 应依赖 Observer 抽象接口，而不是具体观察者。

5. 适配器模式的核心是修改被适配类，使它直接符合客户端需要的接口。（ ）

答案：错

解析： Adapter 通常不修改已有类，而是包装已有类并转换接口。

6. 适配器模式常用于旧系统复用、第三方库接口不兼容等场景。（ ）

答案：对

解析： 第三方 SDK 适配是典型场景。

7. 组合模式适合处理树形结构，并允许客户端统一处理单个对象和组合对象。（ ）

答案：对

解析： Composite 统一处理 Leaf 和 Composite。

8. 组合模式只能用于图形界面，不能用于文件系统或组织结构。（ ）

答案：错

解析： Composite 可用于文件系统、菜单、组织结构等树形结构。

9. 命令模式可以把请求封装成对象，从而支持撤销、排队、日志记录等操作。（ ）

答案：对

解析： Command 把请求封装为对象。

10. 命令模式中，Invoker 通常直接调用 Receiver 的具体业务方法，而不需要 Command 对象。（ ）

答案：错

解析： Invoker 通常调用 Command，Command 再调用 Receiver。

11. 装饰器模式可以在不修改原类的情况下动态增加对象功能。（）

答案：对

解析：Decorator 动态增强对象功能。

12. 装饰器模式通常用于解决“功能组合太多导致子类爆炸”的问题。（）

答案：对

解析：Decorator 常用于避免继承组合爆炸。

13. 单例模式可以保证一个类只有一个实例，并提供全局访问点。（）

答案：对

解析：Singleton 保证唯一实例和全局访问点。

14. 因为单例模式使用方便，所以所有全局使用的对象都应该设计成单例。（）

答案：错

解析：Singleton 容易造成全局状态和隐藏依赖，不应滥用。

15. 抽象工厂模式适合创建一族相关对象，并保证产品族的一致性。（）

答案：对

解析：Abstract Factory 创建一族相关产品。

16. 抽象工厂模式主要用于封装可替换算法。（）

答案：错

解析：封装可替换算法是 Strategy。

17. Visitor 访问者模式适合对象结构较稳定，但需要经常增加新操作的场景。（）

答案：对

解析：Visitor 适合稳定对象结构上增加新操作。

18. Visitor 模式在频繁增加新的元素类型时非常方便，不需要修改已有访问者。（）

答案：错

解析：Visitor 增加新操作方便，但增加新元素类型不方便，因为要修改访问者接口和所有访问者。

19. Iterator 模式的目标是遍历聚合对象而不暴露其内部存储结构。（）

答案：对

解析：Iterator 隐藏集合内部结构。

20. DI 是 GoF 23 种设计模式之一，和 Strategy、Observer 一样属于行为型模式。（）

答案：错

解析：DI 不是 GoF 23 种设计模式，它是依赖注入技术，常和 DIP、TDD、Mock 一起使用。

B. 重构 / Bad Smell 判断题

21. Duplicated Code 表示多个地方出现相同或相似代码，常见重构方式包括 Extract Method 或 Extract Class。()

答案：对

解析：重复代码常通过提取公共逻辑解决。

22. Long Method 一般说明方法承担了过多细节，常见重构方式是 Extract Method。()

答案：对

解析：长方法常用 Extract Method 拆分。

23. 一个方法越长，说明它封装得越完整，通常越容易维护。()

答案：错

解析：方法过长通常降低可读性和可测试性。

24. Public Data Members 会破坏封装，使外部代码可以直接修改对象内部状态。()

答案：对

解析：public 字段破坏封装。

25. Long Parameter List 可以通过 Introduce Parameter Object 将相关参数封装为对象。()

答案：对

解析：长参数列表可封装为参数对象。

26. Data Clumps 指一组相关数据经常一起出现，却没有被组织成类。()

答案：对

解析：相关数据一起出现却没有组织成类，就是数据泥团。

27. Primitive Obsession 指过度使用基本类型表示业务概念，例如用 double 和 String 分别表示金额和币种。()

答案：对

解析：金额 Money 这类业务概念不应长期散落为基本类型。

28. Feature Envy 指一个方法大量访问另一个类的数据，说明该方法可能更适合移动到另一个类中。()

答案：对

解析：Feature Envy 常用 Move Method 处理。

29. Large Class 往往意味着类的职责过多，可能违反单一职责原则。（）

答案：对

解析： 大类通常职责过多。

30. Switch Statements 永远都是错误设计，任何情况下出现 switch 都必须改成设计模式。（）

答案：错

解析： switch 是坏味道的信号，但不是任何情况下都必须消除。

31. Comments Used to Explain Difficult Code 表示代码质量高，因为程序员写了很多注释。（）

答案：错

解析： 用注释解释难懂代码通常说明代码需要重构。

32. Poor Name 指方法名或变量名不能清楚表达其意图，例如 calc、dolt、x 等。（）

答案：对

解析： Poor Name 是命名不能表达意图。

33. Refused Bequest 指子类只使用父类的一小部分行为，甚至拒绝父类行为。（）

答案：对

解析： 子类拒绝父类行为是继承关系不合理的信号。

34. Global Variables 可能导致隐藏依赖和测试之间互相影响。（）

答案：对

解析： 全局变量会造成隐藏共享状态。

35. Speculative Generality 指为了未来可能出现但当前并不存在的需求，提前设计过度抽象。（）

答案：对

解析： Speculative Generality 是为了猜测未来需求而过度设计。

C. 测试驱动开发 TDD 判断题

36. TDD 的典型流程是 Test → Code → Refactor。（）

答案：对

解析： TDD 典型循环是 Test-Code-Refactor。

37. TDD 是先写完整功能代码，再补充测试用例。（）

答案：错

解析： TDD 是先写测试，再写代码。

38. TDD 不只是自动测试技术，也可以看成一种设计技术。（）

答案：对

解析： TDD 通过测试倒逼接口和行为设计。

39. 要让组件容易测试，通常需要能够控制其输入和内部状态，并观察其输出。（）

答案：对

解析： 可测试设计要求能控制输入并观察输出。

40. 方法内部直接调用 `new Random()` 生成随机数，会使测试难以稳定控制输入。（）

答案：对

解析： 随机数不可控会导致测试不稳定。

41. 方法内部直接调用 `LocalDate.now()` 获取当前日期，不会影响测试稳定性。（）

答案：错

解析： 当前时间不可控，测试会依赖运行日期。

42. 直接连接真实数据库、网络或外部支付系统，通常会使单元测试变慢且不稳定。（）

答案：对

解析： 外部系统会降低单元测试稳定性。

43. Mock Object 可以用来替代真实数据库、邮件系统、支付接口等不可控依赖。（）

答案：对

解析： Mock 用于替代不可控依赖。

44. 如果 `OrderService` 内部直接 `new EmailSender`，那么测试时很难替换成 `MockSender`。（）

答案：对

解析： 内部直接 `new` 具体依赖会让测试难替换。

45. 依赖注入可以提高可测试性，因为测试时可以注入假的依赖对象。（）

答案：对

解析： DI 可以注入 Mock / Fake。

46. 业务逻辑直接使用 `System.in` 读取用户输入，通常不利于自动化测试。（）

答案：对

解析： `System.in` 需要人工输入，不利于自动化。

47. 业务逻辑直接 `System.out.println` 输出结果，比返回 `Result` 对象更适合单元测试断言。（）

答案：错

解析：返回 Result 对象更容易断言。

48. 好的单元测试应该尽量快速、独立、可重复，并且没有副作用。（）

答案：对

解析：好测试应快速、独立、可重复、无副作用。

49. 单元测试之间依赖固定执行顺序，可以提高测试的可靠性。（）

答案：错

解析：单元测试之间不应依赖顺序。

50. 修复 bug 时，可以先写一个暴露该 bug 的测试，再修改代码使测试通过，这符合 TDD 思路。（）

答案：对

解析：先写暴露 bug 的测试再修复，符合 TDD。

D. SOLID 设计原则判断题

51. SRP 的含义是一个类只能有一个方法。（）

答案：错

解析：SRP 不是一个类只能有一个方法。

52. SRP 更准确的说法是：一个类应该只有一个改变的理由。（）

答案：对

解析：SRP 是一个类应该只有一个改变理由。

53. Rectangle 类同时包含 area() 和 draw(), 可能违反 SRP, 因为它同时承担几何计算和 GUI 绘制职责。（）

答案：对

解析：area 属于几何计算，draw 属于图形显示。

54. OCP 指软件实体应该对扩展开放，对修改关闭。（）

答案：对

解析：OCP 是对扩展开放，对修改关闭。

55. 如果新增一种订单折扣类型必须修改 OrderService 的 if / else 代码，通常违反 OCP。（）

答案：对

解析：新增折扣必须改旧代码，通常违反 OCP。

56. OCP 的意思是系统上线后永远不能修改任何代码。（）

答案：错

解析： OCP 不是永远不能改任何代码，而是希望通过扩展减少对稳定代码的修改。

57. LSP 只要求子类在语法上继承父类即可，不关心行为是否能替换。（）

答案：错

解析： LSP 关注行为替换，不只是语法继承。

58. LSP 关注的是：凡是使用父类对象的地方，替换成子类对象后程序行为仍应正确。（）

答案：对

解析： 这是 LSP 的核心。

59. Square extends Rectangle 一定符合 LSP，因为数学上正方形是矩形。（）

答案：错

解析： 程序行为上 Square 可能不能替换 Rectangle。

60. 如果子类重写父类方法后抛出 UnsupportedOperationException，通常说明可能违反 LSP 或 ISP。（）

答案：对

解析： 抛异常说明子类或实现类不真正支持该行为。

61. ISP 要求接口尽量大，这样所有客户端都能使用同一个统一接口。（）

答案：错

解析： ISP 不是大接口，而是小而专的接口。

62. ISP 要求接口细粒度、面向特定客户端，客户端不应依赖自己不需要的方法。（）

答案：对

解析： 客户端只依赖自己需要的接口。

63. Worker 接口同时包含 work() 和 eat()，RobotWorker 被迫实现 eat() 并抛异常，可能违反 ISP。（）

答案：对

解析： Robot 不需要 eat，却被迫实现，说明接口过胖。

64. DIP 要求高层模块直接依赖低层具体类，以便调用具体功能。（）

答案：错

解析： DIP 正好相反。

65. DIP 的核心是依赖抽象，而不是依赖具体实现。（）

答案：对

解析： DIP 是 depend on abstractions, not concretions。

66. OrderService 直接 new EmailSender，通常违反 DIP，也不利于测试。（）

答案：对

解析： 高层业务类依赖具体低层实现，违反 DIP。

E. 领域模型判断题

67. 静态领域模型主要描述业务中的概念类、属性以及类之间的关系，而不是具体算法流程。（）

答案：对

解析： 领域模型描述业务概念和静态关系。

68. 领域模型就是数据库表设计，两者完全等价。（）

答案：错

解析： 领域模型不是数据库表设计，虽然以后可以映射到表。

69. 需求中出现的所有名词都必须建成概念类。（）

答案：错

解析： 需求名词要筛选，不是所有名词都建类。

70. 适合建成概念类的对象通常需要被长期保存、查询、统计，或者有自己的属性和关系。（）

答案：对

解析： 长期保存、查询、统计、有属性、有关系，是概念类候选标准。

71. “通知报修人”一定要建成“通知”概念类，即使系统不需要保存通知记录。（）

答案：错

解析： 如果只是动作，不保存记录，不一定建类。

72. 如果系统要求记录通知时间、通知方式和通知状态，则可以考虑建立“通知记录”概念类。（）

答案：对

解析： 需要保存通知信息时，可以建通知记录。

73. 当两个概念类之间的关系本身有属性时，通常应该把这个关系提升为中间类。（）

答案：对

解析： 关系有属性时常抽中间类。

74. 维修工人参与某次调度时需要记录角色、接收时间和完成状态，可以建立“派工明细”作为中间类。（）

答案：对

解析： 角色、接收时间、完成状态属于“某维修工参与某调度”的关系。

75. 一次维修活动使用材料时，如果需要记录材料数量和成本，可以建立“材料使用记录”。（）

答案：对

解析：数量、成本属于“某活动使用某材料”的关系。

76. 一次报修的总工时可以由活动记录的开始时间和结束时间统计得到，因此不一定必须单独建“总工时”类。（）

答案：对

解析：总工时通常可以计算得出。

77. 维修工人是否空闲通常可以由排班记录、不可用记录和活动中的调度记录推导，而不一定是普通静态属性。（）

答案：对

解析：空闲状态通常是动态推导结果。

78. 多重性只需要看对象最终完成后的状态，不需要考虑对象生命周期中的临时状态。（）

答案：错

解析：多重性要考虑对象生命周期中的合法状态。

79. 报修单刚创建时可能还没有评价，因此报修单到评价单可以是 0..1。（）

答案：对

解析：报修单可能未完成或未评价，所以评价是 0..1。

80. 领域模型中，提交、选择、点击这类动作通常优先建成类，而报修单、调度记录这类名词通常不需要建类。（）

答案：错

解析：动作通常不是概念类；报修单、调度记录才是典型概念类。

二、判断题高频陷阱总结

1. 绝对化表述通常危险

看到下面词语要警惕：

一定、必须、只能、所有、永远、任何情况下、完全等价

例如：

所有 Bad Smell 都必须用设计模式解决。

这是错的。很多 Bad Smell 只需要 Extract Method、Rename、Move Method。

2. 概念偷换很常见

常见偷换：

错误说法	正确理解
SRP = 一个类只能有一个方法	SRP = 一个类只有一个改变理由
OCP = 永远不能修改代码	OCP = 尽量通过扩展减少修改旧代码
LSP = 语法上继承即可	LSP = 行为上能替换
DIP = 高层依赖低层具体类	DIP = 依赖抽象
领域模型 = 数据库表	领域模型是业务概念模型

3. 设计模式题看“目的”

关键词	模式
多种算法可替换	Strategy
状态变化通知多个对象	Observer
接口不兼容但要复用	Adapter
树形结构，整体部分统一	Composite
请求封装成对象，撤销/排队	Command
动态增加功能	Decorator
全局唯一实例	Singleton
创建一族相关产品	Abstract Factory
稳定对象结构上增加操作	Visitor
遍历集合不暴露结构	Iterator

4. SOLID 题看“违反表现”

现象	原则
一个类职责太多	SRP
新增功能要改旧代码	OCP

现象	原则
子类不能替换父类	LSP
接口太大，被迫实现无关方法	ISP
高层模块直接依赖具体类	DIP

5. TDD 题看“可控和可观察”

难以自动化测试的典型来源：

随机数、系统时间、数据库、文件、网络、邮件、支付、System.in、System.out、GUI、全局变量、静态状态、Thread.sleep

修改方向：

抽接口、依赖注入、Mock / Stub / Fake、返回 Result、分离业务逻辑和外部资源

6. 领域模型题看“概念是否需要记录”

适合建类：

报修单、调度记录、活动记录、维修工人、故障类别、评价单、投诉单、巡检任务、巡检记录、设施、维修材料、材料使用记录、费用单、支付记录、绩效考核

通常不是类，除非需要保存记录：

提交、选择、点击、通知、统计、计算、调度动作本身

如果系统要求保存它们的时间、状态、内容，则可以建成：

提交记录、通知记录、统计报表、调度记录